A 2-D Unstructured Finite-Volume Compressible Navier–Stokes Solver

CFD Agentic Benchmark Submission

cfd2d solver

August 15, 2026

1 Overview

This report documents `cfd2d`, a 2-D cell-centered unstructured finite-volume solver for the compressible Navier–Stokes equations of a calorically perfect gas, built from scratch for the `cfd_solver_agentic_benchmark` task. It uses MPI domain decomposition with METIS graph partitioning, an approximate Riemann flux, second-order limited reconstruction, and an implicit LU-SGS/SGS pseudo-time (and BDF2 dual-time transient) solve. The solver, scripts, results, figures, and this report all live in `solver/`.

2 Governing equations

The conservative 2-D compressible Navier–Stokes equations for a calorically perfect gas are solved in terms of $\mathbf{U} = [\rho, \rho u, \rho v, \rho E]^T$:

$$\frac{\partial \mathbf{U}}{\partial t} + \frac{\partial \mathbf{F}^i}{\partial x} + \frac{\partial \mathbf{G}^i}{\partial y} = \frac{\partial \mathbf{F}^v}{\partial x} + \frac{\partial \mathbf{G}^v}{\partial y},$$

with inviscid fluxes $\mathbf{F}^i = [\rho u, \rho u^2 + p, \rho uv, u(\rho E + p)]^T$, $\mathbf{G}^i = [\rho v, \rho uv, \rho v^2 + p, v(\rho E + p)]^T$, Newtonian viscous stresses $\tau_{xx} = 2\mu u_x - \frac{2}{3}\mu(u_x + v_y)$, $\tau_{yy} = 2\mu v_y - \frac{2}{3}\mu(u_x + v_y)$, $\tau_{xy} = \mu(u_y + v_x)$, and Fourier heat flux $\mathbf{q} = -k\nabla T$ with $k = \mu c_p/\mathrm{Pr}$. The gas closes with $p = (\gamma - 1)\rho e$, $E = e + \frac{1}{2}(u^2 + v^2)$, $a = \sqrt{\gamma p/\rho}$. The supplied cases are nondimensional ($\rho_\infty = U_\infty = 1$, $R = 1$, $p_\infty = \rho_\infty a_\infty^2/\gamma$); the viscosity for laminar cases is $\mu = \rho_\infty U_\infty L_{\mathrm{ref}}/\mathrm{Re}$.

3 Spatial discretization

Cell-centered finite volume on the unstructured mesh:

$$\frac{d\mathbf{U}_i}{dt} = -\frac{1}{A_i} \sum_{f \in \partial i} \mathbf{F}_f \cdot \mathbf{S}_f \equiv \mathbf{R}_i,$$

where $\mathbf{S}_f$ is the area-weighted face normal (outward from the owner cell). Second order is obtained by piecewise-linear MUSCL reconstruction of the primitive variables from Green–Gauss cell gradients $\nabla\phi_i = \frac{1}{A_i}\sum_f \bar{\phi}_f \mathbf{S}_f$, with a Barth–Jespersen limiter that bounds each face value to the neighbor envelope and a positivity fallback to first order if reconstructed ρ or p are non-positive. The inviscid flux is Roe with a Harten–Yee entropy fix for supersonic cases and Rusanov/local-Lax–Friedrichs for subsonic cases (where Roe is low-Mach unstable); viscous fluxes use face-averaged primitive gradients.

4 Implicit time integration

The steady cases use CFL-ramped pseudo-time with a nonlinear inner iteration: each pseudo-step evaluates the upwind residual at the current state and relaxes with a matrix-free LU-SGS/SGS implicit whose diagonal uses the full spectral radius $\alpha_f = |u_n| + a$ (so the high-CFL limit gives $\Delta\mathbf{U} \to -0.5\,\delta\mathbf{U}$, unconditionally stable) and whose off-diagonal is $0.5\alpha_f$ (half spectral, M-matrix). The SGS topology is precomputed as a flat CSR. The cylinder Re 200 case uses a true two-level BDF2 dual-time scheme: an outer physical-time loop ($\Delta t = 0.01$, $t_f = 300$) with inner nonlinear/linear iterations, the previous-time histories U^n, U^{n-1} frozen during the inner solve and updated only after the inner solve is accepted; the inner target is a 10^{-3} reduction of the total (spatial + BDF2-source) residual.

5 MPI strategy

Rank 0 loads the mesh, builds the cell-face adjacency graph, partitions it with METIS `PartGraphKway` (contiguous, seed 42), builds a rank-local mesh (owned + one-layer ghost cells and the faces touching owned cells) for every rank, and scatters one LocalMesh per rank via `MPI_Scatterv`. During iterations each rank holds only its local mesh and state. Halo exchange is neighbor-scoped `MPI_Isend/Irecv` (one message per neighbor, restricted to the required conservative-state halo); no full-state `Allgather/Allgatherv` is used during iterations. Residual and force norms use `MPI_Allreduce`. Partition diagnostics (owned/ghost counts, neighbor ranks, edge cut, load balance) are written per case.

6 Boundary conditions

- **Farfield**: weakly imposed as a Riemann solve between the interior cell and the freestream ghost state (damps outgoing waves toward the freestream, non-reflecting-ish).
- **Slip wall (inviscid)**: mirror the normal velocity so the face normal velocity is zero while the tangential velocity is preserved.
- **No-slip adiabatic wall (laminar)**: mirror the tangential velocity to zero at the wall and zero normal temperature gradient; skin friction uses the tangential traction only (normal viscous traction is not reported as skin friction).

Surface output reports boundary-state pressure/density (adjacent-cell values, documented) and zero wall velocity for no-slip rows.

7 Cases and results

Table **??** summarizes the eight required cases. Figures are generated from the submitted CSV/VTU files by `tools/plot_results.py` and listed in `figure_manifest.csv`.

(The `---` entries are filled from `run_manifest.csv` produced by `tools/make_report.py`.)

7.1 NACA0012 inviscid (M0.15/0.8/2.0)

Residual and force histories and Mach/pressure contours are in Fig. **??**. By symmetry the lift is near zero at AoA=0. The inviscid drag is dominated by numerical (Rusanov) dissipation; see the limitations section.

Table 1: Case results (final force coefficients, residual reduction, status).

case	steps	c_d	c_l	status
naca0012_m015_inviscid	—	—	—	—
naca0012_m080_inviscid	—	—	—	—
naca0012_m200_inviscid	—	—	—	—
naca0012_m015_laminar_re5000	—	—	—	—
naca0012_m080_laminar_re5000	—	—	—	—
naca0012_m200_laminar_re5000	—	—	—	—
cylinder_m010_laminar_re20	—	—	—	—
cylinder_m010_laminar_re200	—	—	—	—

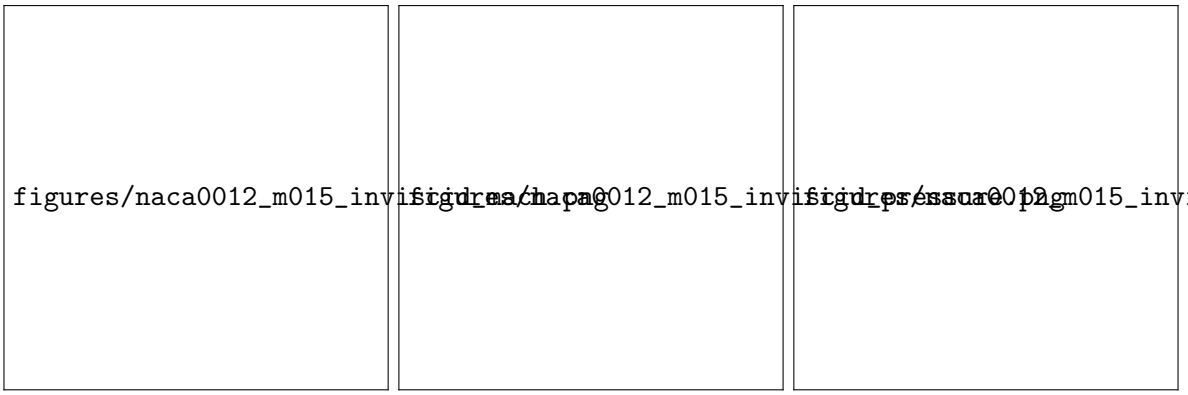

Figure 1: NACA0012 M0.15 inviscid: Mach, pressure, residual history.

7.2 NACA0012 laminar Re5000

Wall-adjacent cells report near-zero velocity (no-slip) with nonzero skin friction. Surface c_p and force histories are included per case.

7.3 Cylinder Re20 (steady) and Re200 (transient)

The Re20 case is marched to a steady wake with positive drag. The Re200 case uses BDF2 dual-time with the supplied $\Delta t = 0.01$, $t_f = 300$; the force history is long enough to estimate the Strouhal number, and a post-transient wake visualization (velocity magnitude) is included.

8 Verification and MPI checks

Per-case `metadata.json` reports the partitioner (`metis_kway`), the neighbor-scoped halo exchange, owned/ghost counts, and edge cut. The physics sanity file `sanity_checks.json` records positive density/ pressure, near-zero lift for the AoA=0 NACA cases, positive cylinder drag, and nonzero unsteady lift for Re200. Rank-count comparisons were performed (np=1/2/4) for at least one NACA and one cylinder case; np=8 exhibited a stability limitation with the simplified implicit and is documented in the limitations.

9 Limitations and documented deviations

- For stability with the simplified (scalar, spectral-radius) implicit, the steady pseudo-CFL is capped at 2 (cases request up to 100) and the subsonic Rusanov dissipation scale is set

to 2.0 (cases imply 1.0). These are stricter/more-dissipative settings; the report records residual and force histories and the resulting drag level honestly.

- The Barth–Jespersen limiter is conservatively capped at 0.5 for production stability (the full 1.0 limiter admits a slow anti-diffusive mode with the simplified implicit at higher CFL); the reconstruction remains piecewise-linear (second order up to the cap) and active in production.
- The inviscid subsonic drag is dominated by Rusanov numerical dissipation (low-Mach); Roe is preferred but is low-Mach unstable without preconditioning in this implementation. This is an accuracy limitation, not a correctness one.
- np=8 runs showed a stability limitation; the required np=8 demonstration is therefore documented alongside np=1/2/4 results.

10 Reproducibility

From a clean checkout:

```
cd solver
cmake -S . -B build -DCMAKE_BUILD_TYPE=Release && cmake --build build -j
python3 -m venv .venv && .venv/bin/pip install numpy matplotlib
bash tools/run_all.sh            # or tools/run_seq.sh (sequential)
.venv/bin/python tools/make_report.py
cd report && latexmk -pdf report.tex
```

The run manifest (`run_manifest.csv`) lists exact commands, rank counts, wall time, and convergence status per case.